Assignment-3

MCA – Machine Learning (MCA201T)

NAME: SRISHTI MALIK ROLL NO. : 253028081

Question 1 :-Define Supervised Learning. Explain the key difference between supervised learning and unsupervised learning with one example of each.

Answer:

Supervised Learning is a type of machine learning in which a model is trained on a **labelled dataset**, i.e., a collection of examples (x_i, y_i) where x_i is the input (feature vector) and y_i is the known correct output (label or target). The algorithm learns a mapping function y = f(x) from inputs to outputs by minimising the error between its predictions and the true labels. Once trained, the function is used to predict the output for new, unseen inputs. It is called 'supervised' because the known labels act like a teacher that guides the learning.

Key difference between supervised and unsupervised learning

Basis	Supervised Learning	Unsupervised Learning
Data used	Labelled data (inputs with known outputs)	Unlabelled data (inputs only)
Goal	Learn a mapping to predict the output for new data	Discover hidden structure, patterns or groups in data
Feedback	Direct feedback: prediction error against true labels	No labels, hence no direct feedback
Typical tasks	Classification and Regression	Clustering, association rules, dimensionality reduction
Evaluation	Easy: accuracy, precision, MSE, etc. against known labels	Harder: silhouette score, inertia, domain judgement
Common algorithms	Linear/Logistic Regression, Decision Trees, SVM	K-Means, Hierarchical clustering, PCA, Apriori

Example of supervised learning: Spam email detection. The model is trained on emails already labelled 'spam' or 'not spam' and then classifies new incoming emails.

Example of unsupervised learning: Customer segmentation. A retailer applies K-Means clustering to unlabelled purchase data to group customers with similar buying behaviour; no predefined groups are given.

Question 2:-*What are the two main types of supervised learning problems? Clearly distinguish between them and give one real-world example for each type.*

Answer:

The two main types of supervised learning problems are **Classification** and **Regression**. They differ in the nature of the target variable that is to be predicted.

Basis	Classification	Regression
Output (target)	Discrete / categorical class label	Continuous numeric value
Goal	Assign an input to one of a set of predefined classes	Predict a quantity as a real number
Sub-types	Binary, multi-class, multi-label	Simple, multiple, polynomial regression
Decision / fitted curve	A decision boundary that separates classes	A best-fit line or curve through the data
Algorithms	Logistic Regression, Decision Tree, SVM, k-NN, Naive Bayes	Linear Regression, Ridge/Lasso, Regression Trees, SVR
Evaluation metrics	Accuracy, Precision, Recall, F1-score, ROC-AUC	MSE, RMSE, MAE, R^2
Example	Email spam detection (Spam / Not Spam)	House price prediction

Real-world example of classification: A bank predicts whether a loan applicant will 'default' or 'not default' using income, credit score and loan amount. The output is one of two categories.

Real-world example of regression: A real-estate website predicts the selling price of a house (for example ₹52.5 lakh) from its area, number of bedrooms and location. The output is a continuous number.

Question 3:- *Explain the concept of Linear Regression. Write the equation of a simple linear regression model and define the terms (dependent variable, independent variable, intercept, and slope). What is the role of the residual/error term?*

Answer:

Concept

Linear Regression is a supervised learning algorithm used to predict a continuous dependent variable from one or more independent variables by fitting a straight line (or hyperplane) to the data. It assumes a linear relationship between the inputs and the output. The best-fit line is found by the **method of Ordinary Least Squares (OLS)**, which minimises the sum of squared differences between the actual and predicted values.

Equation of simple linear regression

$$y = \beta_0 + \beta_1 x + \varepsilon$$

The predicted (fitted) value is $\hat{y} = \beta_0 + \beta_1 x$. The terms are:

- **Dependent variable (y):** the response or target that we want to predict (e.g., marks obtained).
- **Independent variable (x):** the predictor or input feature used to explain y (e.g., hours studied).
- **Intercept (β_0):** the value of y when x = 0; it is the point where the line cuts the y-axis.
- **Slope (β_1):** the change in y for a one-unit increase in x. A positive slope means y increases with x; a negative slope means y decreases.
- **Error term (ε):** the random part of y that the line cannot explain.

Estimating the coefficients (OLS)

$$\beta_1 = \Sigma(x_i - \bar{x})(y_i - \bar{y}) \,/\, \Sigma(x_i - \bar{x})^2 \qquad \beta_0 = \bar{y} - \beta_1 \bar{x}$$

Example: If marks = 35 + 6 × (hours studied), the intercept 35 is the expected mark with zero hours of study, and every extra hour of study increases the expected mark by 6.

Role of the residual / error term

The residual is the difference between the actual and the predicted value: $e_i = y_i - \hat{y}_i$. (ε is the true, unobservable error in the population; the residual e_i is its estimate from the sample.) The error term is important because:

- It accounts for variation in y that is not explained by x, caused by noise, measurement error and omitted variables.
- The model parameters are chosen so that the sum of squared residuals, SSE = Σe_i^2, is minimum.
- Residual plots are used to check the model assumptions: linearity, independence of errors, constant variance (homoscedasticity), and approximately normal errors with zero mean.
- It measures the model's accuracy; smaller residuals indicate a better fit.

Question 4:- *How does Logistic Regression differ from Linear Regression? Write the logistic (sigmoid) function and explain why it is suitable for binary classification. What is the decision boundary in logistic regression?*

Answer:

Difference between Logistic and Linear Regression

Basis	Linear Regression	Logistic Regression
Problem type	Regression (continuous output)	Classification (mainly binary output)
Output	Any real value ($-\infty$ to $+\infty$)	Probability between 0 and 1, converted to a class
Function used	Straight line: $y = \beta_0 + \beta_1 x$	Sigmoid applied to a linear function
Estimation method	Least squares (minimise MSE)	Maximum likelihood (minimise log-loss / cross-entropy)
Relationship modelled	y is linear in x	Log-odds of y = 1 are linear in x
Evaluation	MSE, RMSE, R^2	Accuracy, Precision, Recall, F1, ROC-AUC

Logistic (sigmoid) function

$$\sigma(z) = 1 / (1 + e^{-z}), \quad \text{where } z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

$$P(y = 1 \mid x) = \sigma(z) \qquad P(y = 0 \mid x) = 1 - \sigma(z)$$

Why it is suitable for binary classification

- The output always lies between 0 and 1 ($\sigma(z) \to 0$ as $z \to -\infty$, $\sigma(z) \to 1$ as $z \to +\infty$, and $\sigma(0) = 0.5$), so it can be interpreted as the probability of the positive class.
- A linear model can give values below 0 or above 1, which are meaningless as probabilities; the sigmoid squashes them into the valid range.
- It is smooth, monotonic and differentiable, so the parameters can be learned by gradient descent.
- The log-odds $\ln(p / (1 - p)) = z$ are linear in the features, which makes the model easy to interpret.

Decision boundary

A new sample is assigned to class 1 if $P(y = 1 \mid x) \geq 0.5$ (the usual threshold), otherwise to class 0. Since $\sigma(z) = 0.5$ exactly when $z = 0$, the **decision boundary** is the set of points where

$$\beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n = 0$$

This is a straight line in two dimensions, a plane in three dimensions and a hyperplane in higher dimensions, so logistic regression has a **linear decision boundary**. For example, if $\beta_0 = -4$, $\beta_1 = 1$ and $\beta_2 = 1$, the boundary is $x_1 + x_2 = 4$; points with $x_1 + x_2 > 4$ are classified as class 1. Changing the threshold (say to 0.3) shifts the boundary, and adding polynomial features makes it non-linear.

Question 5:- *Describe how a Decision Tree works for classification. Explain the concepts of Information Gain and Gini Impurity. Why can decision trees overfit the training data, and how can this problem be reduced?*

Answer:

Working of a Decision Tree

A decision tree is a flowchart-like model in which each **internal node** tests a feature, each **branch** represents the outcome of the test, and each **leaf node** holds a class label. It is built by recursive, greedy, top-down partitioning of the data:

1. Start with the entire training set at the root node.
2. For every feature (and possible split point), measure how much a split reduces impurity using Information Gain or Gini Impurity.
3. Select the feature that gives the best split and divide the data into child nodes.
4. Repeat the process recursively on each child node.
5. Stop when a node is pure, the maximum depth is reached, or too few samples remain; the leaf is labelled with the majority class.

To classify a new sample, it is passed from the root down the tree by following the feature tests until it reaches a leaf, whose class is the prediction. Well-known algorithms are ID3 (Information Gain), C4.5 (Gain Ratio) and CART (Gini Impurity).

Entropy and Information Gain

Entropy measures the impurity or randomness of a set S containing classes with proportions p_i:

$$\text{Entropy}(S) = -\Sigma\, p_i \log_2(p_i)$$

Entropy is 0 for a pure node and is maximum (1 for two classes) when the classes are equally mixed. **Information Gain** is the reduction in entropy achieved by splitting S on attribute A:

$$\text{IG}(S, A) = \text{Entropy}(S) - \Sigma\, (|S_v| \,/\, |S|) \times \text{Entropy}(S_v)$$

The attribute with the highest Information Gain is chosen for the split.

Gini Impurity

Gini Impurity is the probability that a randomly chosen sample from a node would be misclassified if it were labelled according to the class distribution in that node:

$$\text{Gini}(S) = 1 - \Sigma\, p_i^2$$

It is 0 for a pure node and has a maximum of 0.5 for a two-class problem. The split that gives the lowest weighted Gini of the child nodes is selected (CART).

Example: A node with 9 'Yes' and 5 'No' samples has Entropy = $-(9/14)\log_2(9/14) - (5/14)\log_2(5/14) \approx 0.940$ and Gini = $1 - (9/14)^2 - (5/14)^2 \approx 0.459$.

Why decision trees overfit

If allowed to grow fully, a tree keeps splitting until every leaf is pure, so it memorises the training data including noise and outliers. This produces a very deep, complex tree with high variance: training accuracy is very high but accuracy on unseen data is poor. Small changes in the data can also produce a very different tree.

How to reduce overfitting

- **Pre-pruning (early stopping):** limit max_depth, set a minimum number of samples to split (min_samples_split) or per leaf (min_samples_leaf), or require a minimum impurity decrease.
- **Post-pruning:** grow the full tree and then remove branches that do not improve validation performance (e.g., cost-complexity pruning).
- **Ensemble methods:** use Random Forest or Gradient Boosting instead of a single tree.
- **Cross-validation** to tune hyperparameters, and **more training data** or removal of noisy/irrelevant features.

Question 6:- *What are Ensemble Methods? Differentiate between Bagging and Boosting with suitable examples of algorithms that use each technique. Why do ensemble methods generally perform better than a single model?*

Answer:

Ensemble methods

Ensemble methods combine the predictions of several individual models (called base learners or weak learners) to produce a single, more accurate and more stable final prediction. The predictions are combined by majority voting (classification) or averaging (regression). The three main techniques are **Bagging, Boosting and Stacking**.

Bagging vs Boosting

Basis	Bagging (Bootstrap Aggregating)	Boosting
Training style	Models are trained in parallel and independently	Models are trained sequentially; each corrects the previous one
Data sampling	Random bootstrap samples (with replacement) of the training set	Whole data is used, but misclassified samples get higher weight (or the next model fits the residuals)
Main aim	Reduce variance (overfitting)	Reduce bias (underfitting), and also variance
Combination	Equal-weight majority vote / average	Weighted vote / weighted sum of learners
Base learners	Strong, high-variance learners (deep trees)	Weak learners (shallow trees / stumps)
Risk	Robust to noise and overfitting	Can overfit noisy data if too many rounds are used
Examples	Bagged Decision Trees, **Random Forest**	**AdaBoost, Gradient Boosting, XGBoost, LightGBM**

Random Forest is bagging of decision trees with an additional random selection of features at each split. **AdaBoost** raises the weights of misclassified samples so that the next learner focuses on them, while **Gradient Boosting** fits each new tree to the residual errors (negative gradient of the loss) of the current ensemble.

Why ensembles perform better than a single model

- **Variance reduction:** averaging many models cancels out their individual random errors. If n models have independent errors with variance σ^2, the variance of their average is σ^2/n.
- **Bias reduction:** boosting combines many weak learners into a strong one that can capture complex patterns.
- **Robustness:** a single model may be a poor choice for a particular dataset; the ensemble is less affected by the mistakes, noise and outliers that mislead one model.
- **Better generalisation:** different models make different errors, so combining diverse models gives lower overall error on unseen data.

This works best when the base learners are individually better than random guessing and are sufficiently diverse (make different errors).

Question 7:- *Explain the working principle of Support Vector Machines (SVM) for binary classification. What are support vectors and the margin? Briefly describe the role of the kernel trick in SVM.*

Answer:

Working principle

A Support Vector Machine finds the **optimal separating hyperplane** that divides the two classes with the **maximum possible margin**. For labels $y_i \in \{-1, +1\}$, the hyperplane is $w \cdot x + b = 0$ and a new point is classified by the sign of $w \cdot x + b$. Among all hyperplanes that separate the classes, SVM chooses the one that is farthest from the nearest points of both classes, because a larger margin gives better generalisation to unseen data.

For perfectly separable data (hard margin), the optimisation problem is:

$$\text{minimise } \tfrac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1 \quad \text{for all } i$$

Margin

The **margin** is the distance between the two parallel boundary hyperplanes $w \cdot x + b = +1$ and $w \cdot x + b = -1$, with the decision hyperplane lying midway between them. Its width is

$$\text{Margin} = 2 \,/\, \|w\|$$

so maximising the margin is equivalent to minimising $\|w\|^2/2$.

Support vectors

Support vectors are the training points that lie closest to the decision boundary, exactly on the margin boundaries ($y_i(w \cdot x_i + b) = 1$). They alone determine the position and orientation of the hyperplane; removing any other point does not change the model, while removing a support vector can.

Soft margin

Real data is rarely perfectly separable. The soft-margin SVM allows some misclassification using slack variables $\xi_i \geq 0$ and minimises $\tfrac{1}{2}\|w\|^2 + C\,\Sigma\xi_i$. The parameter **C** controls the trade-off: a large C gives a narrow margin with fewer training errors (risk of overfitting), while a small C gives a wider margin with more tolerance for errors.

Kernel trick

When the classes are not linearly separable in the original space, the data can be mapped to a higher-dimensional space through a function $\phi(x)$ in which a linear hyperplane can separate them. The **kernel trick** avoids computing this mapping explicitly: SVM depends on the data only through dot products, and a kernel function computes the dot product in the high-dimensional space directly:

$$K(x, z) = \varphi(x) \cdot \varphi(z) \qquad f(x) = \text{sign}\left(\Sigma\, \alpha_i\, y_i\, K(x_i, x) + b \right)$$

This makes it possible to learn non-linear decision boundaries efficiently. Common kernels are:

- **Linear:** $K(x, z) = x \cdot z$
- **Polynomial:** $K(x, z) = (\gamma\, x \cdot z + r)^d$
- **RBF (Gaussian):** $K(x, z) = \exp(-\gamma\|x - z\|^2)$

Question 8:- *List and briefly explain any four evaluation metrics used for classification problems (e.g., Accuracy, Precision, Recall, F1-Score, ROC-AUC). When is Accuracy not a good metric, and which metric would you prefer instead?*

Answer:

Classification metrics are computed from the **confusion matrix**:

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

1. Accuracy

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

The proportion of all predictions that are correct. It is simple but can be misleading on imbalanced data.

2. Precision

$$\text{Precision} = TP / (TP + FP)$$

Out of all instances predicted as positive, the fraction that are actually positive. It matters when false positives are costly (e.g., spam filtering, where a genuine email must not be marked as spam).

3. Recall (Sensitivity / True Positive Rate)

$$\text{Recall} = TP / (TP + FN)$$

Out of all actual positives, the fraction correctly identified. It matters when false negatives are costly (e.g., cancer detection, fraud detection).

4. F1-Score

$$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

The harmonic mean of precision and recall. It gives a single score that balances both and is high only when both are high.

5. ROC-AUC (additional)

The ROC curve plots the True Positive Rate (Recall) against the False Positive Rate (FP / (FP + TN)) at all classification thresholds. The **AUC** is the area under this curve; 1.0 indicates a perfect classifier and 0.5 indicates random guessing. It is independent of any single threshold.

When is Accuracy not a good metric?

Accuracy is unreliable on **imbalanced datasets**, where one class is far more frequent than the other. For example, if only 6 out of 100 transactions are fraudulent, a useless model that always predicts 'not fraud' achieves 94% accuracy but detects no fraud at all (Recall = 0).

Preferred metrics: In such cases **Recall** (when missing a positive case is costly, as in disease or fraud detection), **Precision** (when false alarms are costly), or the **F1-Score** (when a balance is needed) should

be used. ROC-AUC or the Precision–Recall curve is also suitable for comparing models. For the fraud example, Recall and F1-Score are the best choices.

Question 9:- *For regression problems, explain the following evaluation metrics: (a) Mean Squared Error (MSE) (b) Root Mean Squared Error (RMSE) (c) Mean Absolute Error (MAE) (d) R-squared (Coefficient of Determination). Which metric is more sensitive to outliers and why?*

Answer:

Let y_i be the actual value, $\hat{y}_i$ the predicted value, $\bar{y}$ the mean of the actual values and n the number of observations.

(a) Mean Squared Error (MSE)

$$\text{MSE} = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2$$

The average of the squared differences between actual and predicted values. Squaring removes the sign and penalises large errors heavily. Its unit is the square of the target's unit, and a lower value is better.

(b) Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{[\ (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2\ ]}$$

The square root of MSE. It is in the same unit as the target variable, so it is easier to interpret (e.g., 'the prediction is off by about ₹2 lakh on average'), while still penalising large errors.

(c) Mean Absolute Error (MAE)

$$\text{MAE} = (1/n)\ \Sigma\ |y_i - \hat{y}_i|$$

The average of the absolute errors. It is in the same unit as the target and treats all errors in proportion to their size, so it is more robust to outliers.

(d) R-squared (Coefficient of Determination)

$$R^2 = 1 - SS_{res} / SS_{tot} = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$$

R^2 gives the proportion of the variance in the dependent variable that is explained by the model. It usually lies between 0 and 1: $R^2 = 1$ is a perfect fit, $R^2 = 0$ means the model is no better than always predicting the mean, and it can be negative for a model worse than that baseline. For example, $R^2 = 0.85$ means that 85% of the variation in y is explained by the model.

Numerical illustration

Actual y = [3, 5, 7, 9], predicted $\hat{y}$ = [2, 5, 8, 12]. Errors = 1, 0, −1, −3. Then MSE = (1 + 0 + 1 + 9)/4 = 2.75, RMSE = √2.75 ≈ 1.66, MAE = (1 + 0 + 1 + 3)/4 = 1.25. With $\bar{y} = 6$, $SS_{tot} = 20$ and $SS_{res} = 11$, so $R^2 = 1 - 11/20 = 0.45$.

Which metric is more sensitive to outliers?

MSE (and therefore RMSE) is the most sensitive to outliers, because the errors are squared before averaging, so a large error contributes disproportionately. For example, for errors 1, 1, 1, 1 and 10: MAE = 14/5 = 2.8, whereas MSE = (4 + 100)/5 = 20.8 and RMSE ≈ 4.56. The single outlier contributes 100 of the 104 total squared error, but only 10 of the 14 total absolute error. MAE grows only linearly with the error and is therefore more robust. R^2 is also affected by outliers indirectly since it is built on squared errors.

Question 10:- *Discuss any four real-world applications of Supervised Learning. For each application, mention the type of supervised learning problem (classification or regression) and a suitable algorithm that can be used.*

Answer:

1. Spam email detection

- **Problem type:** Binary classification (Spam / Not spam).
- **Suitable algorithm:** Naive Bayes, Logistic Regression or SVM.
- **How it works:** The model is trained on emails labelled as spam or not spam, using features such as word frequencies, sender and links, and then filters new incoming mail.

2. House price prediction

- **Problem type:** Regression (continuous output – price).
- **Suitable algorithm:** Linear Regression, Random Forest Regressor or Gradient Boosting.
- **How it works:** The model learns from past sales with features such as area, number of rooms, location and age of the property, and predicts the price of a new house.

3. Medical diagnosis (e.g., detecting diabetes or classifying a tumour as benign or malignant)

- **Problem type:** Classification.
- **Suitable algorithm:** Decision Tree, Random Forest, SVM or Logistic Regression.
- **How it works:** Patient records or test results (age, blood sugar, BMI, tumour size, etc.) with confirmed diagnoses are used for training; the model then predicts the disease status of new patients, helping doctors in early detection.

4. Credit card fraud detection

- **Problem type:** Binary classification (Fraudulent / Genuine), usually with highly imbalanced data.
- **Suitable algorithm:** Random Forest, XGBoost (Gradient Boosting) or Logistic Regression.
- **How it works:** The model is trained on past transactions labelled as fraud or genuine, using features like amount, location, time and merchant type, and flags suspicious transactions in real time. Recall and F1-score are used to evaluate it.

Other applications include sales and stock-price forecasting (regression), sentiment analysis of reviews (classification), and handwritten digit or face recognition (classification).